

Internship & Job Tracking Dashboard

Table of Contents

1. Introduction
2. Problem Statement
3. System Architecture
4. Database Design
5. Docker Compose Explanation
6. Streamlit Application — Pages & Screenshots
7. CRUD Operations
8. CSV Upload and Export
9. Login and Role Management
10. Analytics Dashboard
11. pgAdmin Verification
12. GitHub Contribution Evidence
13. Challenges and Solutions
14. Conclusion

1. Introduction

This report documents the design, development, and deployment of the **Internship & Job Tracking Dashboard** — a full-stack web application built as the final project submission for the *Tools & Techniques for Data Science* course (CSDS4473) at the University of Central Punjab.

The system was built using **Streamlit** for the frontend, **PostgreSQL** as the persistent relational database, **pgAdmin** for database administration, and **Docker Compose** to orchestrate the entire multi-container environment. All source code is version-controlled through **GitHub** with meaningful commits from every group member.

Team Members

Name	Role	GitHub Username
Ahmed	CRUD Operations + Authentication + Report	@AHMED-IMTIAZ-DS
M. Soban Saqib	Database Design + Schema + Seed Data	@SOBAN-SAQIB
M. Adil Khan	Analytics Dashboard + Docker Deployment	@adil7t11

Tools and Technologies

Tool / Library	Purpose
Python 3.11	Application programming language
Streamlit	Frontend web application and interactive dashboard
PostgreSQL 16	Relational database for persistent storage
pgAdmin 4	Graphical database administration and verification
Docker Desktop	Container runtime on Windows
Docker Compose	Multi-container application orchestration
SQLAlchemy + psycopg2-binary	Database connection layer from Python
pandas	Data processing and CSV handling
Plotly	Interactive charts and visualisations
python-dotenv	Environment variable loading from .env files
Git + GitHub	Version control and collaboration

2. Problem Statement

University departments receive internship and job opportunity records from various companies throughout the academic year. Without a centralised system, faculty members struggle to manage, search, and analyse these records efficiently. Data is scattered across spreadsheets, email threads, and informal documents — making it difficult to identify expired opportunities, detect duplicates, or generate meaningful analytics for student placement.

The core requirements identified for this project were:

- A structured, searchable database of internship and job opportunities.
- An easy-to-use web interface accessible from any browser.
- Role-based access control to differentiate Admin and Viewer functionality.
- Automated detection of duplicate records and deadline alerts.
- Bulk data import via CSV and export of filtered results.
- A visual analytics dashboard to support data-driven decision making.
- A fully containerised, reproducible deployment using Docker Compose.

3. System Architecture

The application follows a three-tier containerised architecture managed entirely through Docker Compose. Each tier runs in its own isolated container connected via a dedicated bridge network named **app_network**.

```
User Browser
|
v HTTP (port 8501)
Streamlit App Container (opportunity_streamlit)
|
v SQLAlchemy / psycpg2 (port 5432 internal)
PostgreSQL Container (opportunity_postgres) <-----> pgAdmin Container (port 5050)
|
v
Docker Named Volume (postgres_data) - Persistent Data Storage
GitHub Repository - Source code, commits, SQL files, README, report
```

The Streamlit container is built from a custom **Dockerfile** that packages the Python application with all its dependencies. The PostgreSQL container uses the official **postgres:16** image. Initialisation SQL scripts (`init.sql` and `seed_data.sql`) are mounted and executed automatically when the container starts for the first time. The named volume **postgres_data** ensures all database records persist even when containers are restarted or rebuilt.

Folder Structure

```
streamlit-postgres-assignment/
|
+-- app/
|   +-- main.py (Home page + login gate)
|   +-- db.py (SQLAlchemy engine + connection)
|   +-- queries.py (All SQL query functions)
|   +-- auth.py (Login, logout, role checking)
|   +-- utils.py (CSV validation, helpers)
|   +-- pages/
|       +-- 1_Add_Opportunity.py
|       +-- 2_View_Search.py
|       +-- 3_Update_Opportunity.py
|       +-- 4_Delete_Opportunity.py
|       +-- 5_Analytics_Dashboard.py
|       +-- 6_CSV_Upload_Export.py
|       +-- 7_Duplicate_Detection.py
|       +-- 8_Deadline_Alerts.py
|       +-- 9_Database_Health_Check.py
```

```
|
+-- database/
| +-- init.sql (CREATE TABLE + indexes)
| +-- seed_data.sql (45 sample records)
|
+-- screenshots/ (Evidence screenshots)
+-- docker-compose.yml
+-- Dockerfile
+-- requirements.txt
+-- .env.example
+-- README.md
+-- report.pdf
```

4. Database Design

The PostgreSQL database **student_opportunities_db** contains one primary table: **opportunities**. The schema was designed to capture all relevant attributes of an internship or job posting, with appropriate constraints to maintain data integrity.

Table Schema — opportunities

```
CREATE TABLE IF NOT EXISTS opportunities (  
    opportunity_id SERIAL PRIMARY KEY,  
    company_name VARCHAR(100) NOT NULL,  
    job_title VARCHAR(150) NOT NULL,  
    category VARCHAR(50) NOT NULL,  
    city VARCHAR(80),  
    country VARCHAR(80),  
    work_mode VARCHAR(30) CHECK (work_mode IN ('Remote', 'Onsite', 'Hybrid')),  
    required_skills TEXT NOT NULL,  
    salary_min NUMERIC(10,2),  
    salary_max NUMERIC(10,2),  
    currency VARCHAR(10) DEFAULT 'PKR',  
    experience_level VARCHAR(50),  
    application_deadline DATE,  
    status VARCHAR(30) DEFAULT 'Open'  
    CHECK (status IN ('Open', 'Closed', 'Expired', 'Shortlisted')),  
    source_link TEXT,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);  
  
-- Performance indexes  
CREATE INDEX IF NOT EXISTS idx_opportunities_category ON opportunities(category);  
CREATE INDEX IF NOT EXISTS idx_opportunities_status ON opportunities(status);  
CREATE INDEX IF NOT EXISTS idx_opportunities_city ON opportunities(city);  
CREATE INDEX IF NOT EXISTS idx_opportunities_company ON opportunities(company_name);  
CREATE INDEX IF NOT EXISTS idx_opportunities_deadline ON  
opportunities(application_deadline);
```

Column Descriptions

Column	Type	Constraint	Description
opportunity_id	SERIAL	PRIMARY KEY	Auto-incrementing unique identifier
company_name	VARCHAR(100)	NOT NULL	Name of the hiring company
job_title	VARCHAR(150)	NOT NULL	Position title
category	VARCHAR(50)	NOT NULL	Job category (Data Science, AI, etc.)

city	VARCHAR(80)	—	City where the job is located
country	VARCHAR(80)	—	Country (defaults to Pakistan)
work_mode	VARCHAR(30)	CHECK	Remote, Onsite, or Hybrid
required_skills	TEXT	NOT NULL	Comma-separated list of required skills
salary_min / max	NUMERIC(10,2)	—	Salary range in specified currency
currency	VARCHAR(10)	DEFAULT 'PKR'	Currency of salary values
experience_level	VARCHAR(50)	—	Internship / Entry / Mid / Senior
application_deadline	DATE	—	Last date to apply
status	VARCHAR(30)	CHECK + DEFAULT	Open, Closed, Expired, or Shortlisted
source_link	TEXT	—	URL to apply or view posting
created_at	TIMESTAMP	DEFAULT NOW()	Record insertion timestamp

Seed Data Summary

45 sample records were inserted covering 8 companies (Systems Ltd, Arbisoft, NetSol Technologies, Folio3, 10Pearls, Contour Software, Txxel, i2c Inc), 5 cities (Karachi, Lahore, Islamabad, Rawalpindi, Peshawar), 5 job categories (Data Science, AI, Web Development, Cyber Security, Software Engineering), all three work modes (Remote, Onsite, Hybrid), and all four statuses (Open, Closed, Expired, Shortlisted).

5. Docker Compose Explanation

Docker Compose defines the entire application environment in a single **docker-compose.yml** file. Three services are declared: **postgres_db**, **pgadmin**, and **streamlit_app**, all connected via a shared bridge network named **app_network**.

docker-compose.yml

```
version: "3.9"

services:
  postgres_db:
    image: postgres:16
    container_name: opportunity_postgres
    environment:
      POSTGRES_DB: student_opportunities_db
      POSTGRES_USER: app_user
      POSTGRES_PASSWORD: app_password
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
      - ./database/init.sql:/docker-entrypoint-initdb.d/01_init.sql
      - ./database/seed_data.sql:/docker-entrypoint-initdb.d/02_seed_data.sql
    restart: unless-stopped
    networks:
      - app_network

  pgadmin:
    image: dpage/pgadmin4:latest
    container_name: opportunity_pgadmin
    environment:
      PGADMIN_DEFAULT_EMAIL: admin@example.com
      PGADMIN_DEFAULT_PASSWORD: admin123
    ports:
      - "5050:80"
    depends_on:
      - postgres_db
    restart: unless-stopped
    networks:
      - app_network

  streamlit_app:
    build: .
    container_name: opportunity_streamlit
```

```
ports:
- "8501:8501"
environment:
DB_HOST: postgres_db
DB_PORT: 5432
DB_NAME: student_opportunities_db
DB_USER: app_user
DB_PASSWORD: app_password
depends_on:
- postgres_db
restart: unless-stopped
networks:
- app_network
volumes:
- ./app:/app/app
volumes:
postgres_data:
networks:
app_network:
driver: bridge
```

Key Docker Compose Concepts

Concept	Usage in This Project
services	Three services defined: postgres_db, pgadmin, streamlit_app
image	postgres:16 and dpage/pgadmin4:latest for pre-built images
build	Used for streamlit_app to build from the project's Dockerfile
container_name	Explicit names: opportunity_postgres, opportunity_pgadmin, opportunity_streamlit
ports	Host:Container mapping — 5432, 5050, 8501
environment	DB credentials passed as environment variables
volumes	postgres_data named volume for persistence; init SQL files mounted
depends_on	pgadmin and streamlit_app wait for postgres_db to start
restart	unless-stopped ensures auto-restart on Docker/system restart
networks	app_network bridge enables service-name-based DNS resolution

Dockerfile

```
FROM python:3.11-slim
WORKDIR /app
```

```
COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY app/ ./app/

EXPOSE 8501

CMD ["streamlit", "run", "app/main.py",
"--server.address=0.0.0.0", "--server.port=8501"]
```

Docker Commands Used

Command	Purpose
docker compose up -d	Start all services in detached background mode
docker compose ps	Check running services and mapped ports
docker compose logs postgres_db	View PostgreSQL container logs
docker compose logs streamlit_app	View Streamlit app container logs
docker compose down	Stop and remove containers, preserving volumes
docker volume ls	List all Docker volumes
docker volume inspect postgres_data	Inspect the PostgreSQL data volume
docker compose down -v	Stop containers AND delete volumes (data loss warning)

6. Streamlit Application — Pages and Functionality

The Streamlit application is structured as a **multipage application** using Streamlit's native pages directory approach. The main entry point `app/main.py` handles authentication and the home dashboard, while nine individual page modules live in `app/pages/`.

[main.py — Home / Introduction](#)

Serves as the application entry point and login gate. Displays project title, team members, tools used, an app guide using `st.tabs`, and the architecture diagram. Shows live database connection status using `st.metric` and `st.success/st.error`. Uses `st.session_state` to manage login state and `st.cache_resource` in `db.py` for efficient connection pooling.

[1_Add_Opportunity.py — Add New Opportunity](#)

Admin-only page with a comprehensive `st.form` containing all 14 fields. Validates required fields (company name, job title, skills), salary logic (min cannot exceed max), and checks for potential duplicates before insertion. Uses `st.selectbox`, `st.text_input`, `st.text_area`, `st.number_input`, `st.date_input`. Displays `st.success` + `st.balloons` on success.

[2_View_Search.py — View and Search](#)

Available to both Admin and Viewer roles. Sidebar filters for category, city, work mode, status, experience level, and salary range with text search across company/title/skills. Displays results in `st.dataframe` with `column_config` formatting. Shows 4 KPI metrics and a record detail view for any selected ID. Includes inline CSV export download button.

[3_Update_Opportunity.py — Update Opportunity](#)

Admin-only page. Loads all records into a `selectbox` dropdown. Displays current record details then presents an `st.form` for editing status, work mode, experience level, deadline, salary, skills, and source link. Validates salary logic before calling `update_opportunity()`.

[4_Delete_Opportunity.py — Delete Opportunity](#)

Admin-only page with a confirmation-before-delete workflow. Shows full record preview, requires the user to check an `st.checkbox` to confirm, then enables the delete button. Uses `st.error` for permanent action warnings and `st.rerun()` to refresh after deletion.

[5_Analytics_Dashboard.py — Analytics Dashboard](#)

Available to all roles. Displays 6 KPI metrics (total, open, shortlisted, remote, internships, companies) and 7 Plotly charts: category pie, status bar, work mode pie, city horizontal bar, salary grouped bar, experience bar, monthly trend line, and skills frequency horizontal bar. Uses `st.plotly_chart` with `use_container_width=True` for responsive layouts.

[6_CSV_Upload_Export.py — CSV Upload / Export](#)

Admin upload tab: accepts CSV files, shows raw preview (first 10 rows), runs `validate_csv()` to check required columns/work_mode/status/empty fields, displays valid rows, then inserts via `bulk_insert()`. Export tab: filter by category/status/work mode, preview up to 20 rows, download as CSV with

st.download_button. Includes template CSV download.

7_Duplicate_Detection.py — Duplicate Detection

Available to all roles; Admin can act on duplicates. Uses a SQL GROUP BY query on (company_name, job_title, city, source_link) with HAVING COUNT(*) > 1. Each duplicate group is shown in an st.expander. Admins can remove all duplicates while keeping the first record.

8_Deadline_Alerts.py — Deadline Alerts

Two tabs: opportunities closing within the next 7 days (Open status, deadline between today and today + 7 days) and expired/closed opportunities. Shows exact days remaining for each closing-soon record. Uses colour-coded st.warning and st.error messaging.

9_Database_Health_Check.py — Database Health Check

Tests PostgreSQL connection using test_connection(), displays version and timestamp. Shows total row count, database size in bytes, and table size via pg_size_pretty. Displays latest inserted record and full table schema from information_schema.columns. Includes a Quick SQL Queries runner with 6 predefined verification queries.

Screenshots

The screenshots folder contains the following evidence captured during live execution of the application:

Screenshot File	Content
1.PNG	Home page of the Streamlit application after login
add_form.png	Add New Opportunity form with all input fields
analytics_1.PNG	Analytics Dashboard — KPI metrics row
analytics_2.PNG	Analytics Dashboard — Category and Status charts
analytics_3.PNG	Analytics Dashboard — Work Mode and City charts
analytics_4.PNG	Analytics Dashboard — Salary and Experience charts
analytics_5.PNG	Analytics Dashboard — Monthly trend and Skills frequency
pgadmin_connection.PNG	pgAdmin connected to opportunity_postgres container
postgres_table.PNG	opportunities table visible in pgAdmin with records

7. CRUD Operations Explanation

All database operations are centralised in **app/queries.py**, which uses SQLAlchemy's `text()` for parameterised queries, preventing SQL injection. The connection layer in **app/db.py** uses `@st.cache_resource` to maintain a single engine instance across Streamlit reruns.

Create — `insert_opportunity()`

```
def insert_opportunity(data: dict):
    sql = text("""
    INSERT INTO opportunities (
    company_name, job_title, category, city, country, work_mode,
    required_skills, salary_min, salary_max, currency,
    experience_level, application_deadline, status, source_link
    ) VALUES (
    :company_name, :job_title, :category, :city, :country, :work_mode,
    :required_skills, :salary_min, :salary_max, :currency,
    :experience_level, :application_deadline, :status, :source_link
    ) RETURNING opportunity_id
    """)
    with get_connection() as conn:
        result = conn.execute(sql, data)
        conn.commit()

    return result.fetchone()[0]
```

Read — `fetch_filtered()`

The `fetch_filtered()` function builds a dynamic WHERE clause based on the filters provided. Each optional parameter adds a condition only if it is not None or empty, using parameterised bindings to prevent injection. The ILIKE operator is used for case-insensitive text search across `company_name`, `job_title`, and `required_skills`.

Update — `update_opportunity()`

Accepts a dictionary of fields to update. Dynamically constructs the SET clause from the dictionary keys, then executes the UPDATE with the `opportunity_id` as the WHERE condition.

Delete — `delete_opportunity()`

```
def delete_opportunity(opportunity_id: int):
    with get_connection() as conn:
        conn.execute(
            text("DELETE FROM opportunities WHERE opportunity_id = :id"),
            {"id": opportunity_id}
        )
        conn.commit()
```

8. CSV Upload and Export

CSV Upload Workflow

- User selects a CSV file via `st.file_uploader` (accepts .csv only).
- Raw file is read with `pandas.read_csv()` and a 10-row preview is shown.
- `validate_csv()` checks for required columns, valid `work_mode` values, valid `status` values, and non-empty `company_name` and `job_title` fields.
- Validation errors are shown as `st.warning` messages.
- Valid rows are displayed in a separate preview dataframe.
- On clicking 'Insert Valid Rows', `dataframe_to_records()` converts rows to dicts and `bulk_insert()` loops through each record calling `insert_opportunity()`.
- Inserted and skipped counts are reported to the user.

CSV Export Workflow

- User applies optional filters: category, status, work mode.
- `fetch_filtered()` retrieves matching records from PostgreSQL.
- Result is shown in a preview dataframe (first 20 rows).
- `st.download_button` triggers download of the full filtered result as a CSV file using `df_to_csv_bytes()` which calls `DataFrame.to_csv(index=False).encode('utf-8')`.
- Export is also available directly from the View & Search page for the current filtered view.

Required CSV Columns

Column Name	Required?	Validation Rule
company_name	Yes	Must not be empty
job_title	Yes	Must not be empty
category	Yes	No restriction enforced in CSV validator
city	Yes	—
country	Yes	—
work_mode	Yes	Must be Remote, Onsite, or Hybrid
required_skills	Yes	—
salary_min / salary_max	No	Numeric; None accepted
currency	No	Defaults to PKR
experience_level	No	—
application_deadline	No	Date string
status	Yes	Must be Open, Closed, Expired, or Shortlisted

source_link	No	URL string
-------------	----	------------

9. Login and Role Management

Authentication is implemented in **app/auth.py** using Streamlit's **session_state**. Two hard-coded user accounts are defined (suitable for an academic demo environment). The login gate in **main.py** prevents any page from loading until the user is authenticated.

User Accounts

Username	Password	Role	Permissions
admin	admin123	Admin	Full access — add, update, delete, upload CSV, manage duplicates
viewer	view123	Viewer	Read-only — view records, search, analytics, deadline alerts

Session State Variables

Variable	Type	Description
st.session_state['logged_in']	bool	True if user is authenticated
st.session_state['username']	str	Active username
st.session_state['role']	str	Admin or Viewer

Role Enforcement Functions

require_login() — called at the top of every page. Redirects to login form and calls `st.stop()` if the user is not authenticated.

require_admin() — called on Add, Update, Delete, and CSV Upload pages. Shows an Access Denied error and calls `st.stop()` if the role is not Admin.

is_admin() — helper used inline to conditionally show admin-only buttons (e.g. duplicate removal buttons in the Duplicate Detection page).

The sidebar always shows the **show_user_info()** component which displays the current username, role badge, and a Logout button that clears `session_state` and calls `st.rerun()`.

10. Analytics Dashboard

The Analytics Dashboard (page 5) is the most data-rich page of the application. It combines six KPI metrics and seven interactive Plotly charts to provide comprehensive insights into the opportunities database.

Key Performance Indicators (6 KPIs)

KPI	Query Source	Description
Total Opportunities	count_rows()	Full count of all records
Open Positions	fetch_all() filtered	Records with status = 'Open'
Shortlisted	fetch_all() filtered	Records with status = 'Shortlisted'
Remote Jobs	fetch_all() filtered	Records with work_mode = 'Remote'
Internships	fetch_all() filtered	Records with experience_level = 'Internship'
Companies	df['company_name'].nunique ()	Distinct companies in dataset

Charts (7 Charts via Plotly)

Chart	Chart Type	Data Source
Opportunities by Category	Donut Pie (px.pie, hole=0.4)	GROUP BY category
Opportunities by Status	Colour-coded Bar (px.bar)	GROUP BY status
Work Mode Distribution	Pie (px.pie)	GROUP BY work_mode
Opportunities by City	Horizontal Bar (px.bar, orientation=h)	GROUP BY city
Average Salary by Category	Grouped Bar (go.Figure + go.Bar x2)	AVG(salary_min/max) GROUP BY category
Experience Level Breakdown	Bar (px.bar)	GROUP BY experience_level
Monthly Posting Trend	Spline Line (px.line)	TO_CHAR(created_at, 'YYYY-MM') GROUP BY month
Most In-Demand Skills	Horizontal Bar (px.bar, Viridis scale)	Split required_skills by ' ', value_counts top 20

11. pgAdmin Verification

pgAdmin is accessible at **http://localhost:5050** using email `admin@example.com` and password `admin123` (as configured in `docker-compose.yml`). When registering a new server in pgAdmin, the host name must be set to **postgres_db** (the Docker service name) rather than `localhost`, since both containers communicate over the `app_network` bridge.

pgAdmin Connection Settings

Setting	Value
Host name / address	postgres_db
Port	5432
Maintenance database	student_opportunities_db
Username	app_user
Password	app_password

Required SQL Verification Queries

```
-- 1. View all records
SELECT * FROM opportunities;

-- 2. Total record count
SELECT COUNT(*) FROM opportunities;

-- 3. Records per category
SELECT category, COUNT(*)
FROM opportunities
GROUP BY category;

-- 4. Records per work mode
SELECT work_mode, COUNT(*)
FROM opportunities
GROUP BY work_mode;

-- 5. Open opportunities only
SELECT *
FROM opportunities
WHERE status = 'Open';

-- 6. Opportunities closing within 7 days
SELECT *
FROM opportunities
WHERE application_deadline <= CURRENT_DATE + INTERVAL '7 days';
```

Screenshots **pgadmin_connection.PNG** and **postgres_table.PNG** in the screenshots folder show pgAdmin successfully connected to the PostgreSQL container and the opportunities table populated with

seed data records.

12. GitHub Contribution Evidence

The project was managed through Git with 15 meaningful commits distributed across all team members throughout the development timeline. Commit messages follow a consistent, descriptive convention covering each major feature or module.

Git Commit History

Commit Hash	Commit Message	Member
fa70cd8	create opportunities table schema	Soban
f21ad36	add sample opportunity records	Soban
ce66dcd	implement PostgreSQL connection layer	Ahmed
78ce001	add CRUD database queries	Ahmed
77ce57f	create database health monitoring page	Adil
89558ed	create analytics dashboard with KPIs	Adil
9be81bf	implement CSV upload and export features	Ahmed
0d3abb4	add duplicate opportunity detection	Ahmed
625e571	implement deadline alert system	Ahmed
4d14317	create Streamlit application structure	Soban
1f4c9b5	implement add opportunity form	Soban
738167e	implement search and filtering module	Ahmed
b5ff6f2	implement opportunity update functionality	Adil
21eb931	add deletion workflow and role based access	Adil
92eeb35	configure Docker deployment and project documentation	Adil

Contribution Summary

Member	Tasks Completed	Commits
Ahmed	DB connection layer, CRUD queries, CSV upload/export, duplicate detection, deadline alerts, search/filter page, authentication	6
M. Soban Saqib	Database schema, seed data SQL, app structure, Add Opportunity form	4
M. Adil Khan	Analytics dashboard, health check page, update/delete pages, Docker configuration, README and documentation	5

GitHub Evidence: Screenshots of the repository home page, commit history page showing all contributors, and the contributors/insights page are available in the screenshots folder. The GitHub repository link is included in the README.md file submitted with this project.

13. Challenges Faced and Solutions

PostgreSQL container not starting

Solution: Checked docker compose logs postgres_db. Found that port 5432 was already bound by a local PostgreSQL installation. Solution: stopped the local service with `net stop postgresql` and restarted the Docker container.

pgAdmin unable to connect to PostgreSQL

Solution: Initially used 'localhost' as the host name in pgAdmin's server registration. This failed because pgAdmin runs inside a Docker container and 'localhost' refers to the pgAdmin container itself, not the host machine. Solution: changed the host to 'postgres_db' (the Docker service name) to use Docker's internal DNS resolution.

Streamlit app crashing on startup

Solution: The streamlit_app container started before PostgreSQL was fully ready despite using `depends_on`. Solution: added `pool_pre_ping=True` to the SQLAlchemy engine and wrapped all database calls in try/except blocks with user-friendly `st.error` messages.

CSV upload failing for some rows

Solution: Some uploaded CSV files had mismatched column names (extra spaces, different capitalisation). Solution: updated `validate_csv()` to check for the exact expected column names and report missing columns clearly to the user before attempting any insertions.

session_state resetting on page navigation

Solution: Streamlit reruns the script on every interaction, which initially caused the login state to reset. Solution: initialised `session_state` variables in `main.py` only when they do not already exist, using the `'not in st.session_state'` guard pattern.

Git push rejected due to diverged histories

Solution: When two members pushed to GitHub simultaneously, pushes were rejected. Solution: team adopted a pull-before-push workflow (`git pull --rebase origin main` before pushing) and resolved merge conflicts using VS Code's built-in diff editor.

Docker volume data loss during development

Solution: Running `docker compose down -v` during debugging deleted all seeded data. Solution: documented clearly in README to use `docker compose down` (without `-v`) to preserve the `postgres_data` volume. Seed data SQL ensures the database can be reproduced.

14. Conclusion

This project successfully demonstrates the full-stack integration of Streamlit, PostgreSQL, pgAdmin, and Docker Compose to build a professional, reproducible data application. All required features outlined in the assignment brief were implemented and are functional.

The **Internship & Job Tracking Dashboard** achieves the following learning outcomes:

- A complete Streamlit multipage application with 9 functional pages.
- Role-based access control separating Admin and Viewer permissions.
- Full CRUD operations against a PostgreSQL database using SQLAlchemy.
- CSV bulk upload with validation and CSV export with filtering.
- Automated duplicate detection using SQL GROUP BY / HAVING logic.
- Deadline alerts for closing-soon and expired opportunities.
- A comprehensive analytics dashboard with 6 KPIs and 7 interactive Plotly charts.
- A fully containerised deployment via Docker Compose with a custom Dockerfile.
- Persistent data storage through a named Docker volume.
- Database verification through pgAdmin with SQL query evidence.
- Collaborative GitHub workflow with 15 meaningful commits from all members.
- A clean, documented README with setup, troubleshooting, and architecture notes.

The system is fully reproducible — any user with Docker Desktop installed can clone the GitHub repository and run `docker compose up -d` to bring the entire application stack online within minutes, with all 45 seed records pre-loaded in the database.

CLO 4 — Demonstrate reproducible and deployable data applications using Docker and Streamlit through an end-to-end integrated project. [Bloom's Level C3 — P5] — SATISFIED